P.D.E.A's
Prof. Ramkrishna More A.C.S. College (Autonomous) Akurdi,Pune-44
Department Of Computer Science
T.Y.B.Sc(CS)
CSMAP-364 Operating System-II and Data Analytics Laboratory
Text Analysis

Natural Language Toolkit (NLTK) : NLTK is a Python Package for performing Natural Language Processing on human language data which is mostly unstructured. It mainly focuses on analyzing textual data.
First Upgrade the pip command:
C:\Users\admin\AppData\Local\Microsoft\WindowsApps\PythonSoftwareFoundation.Python.3.13_qbz5n2kfra8p0\python.exe -m pip install --upgrade pip
Installation of NLTK:

1. pip install nltk
2. nltk.download()
3. nltk.download('punkt')
4. nltk.download('punkt_tab')
5. nltk.download('all')

Pre-processing steps for text analytics using NLTK:
Tokenization: It is the first step to perform text analytics. Tokenization means breaking down a textual paragraph into small chunks such as words or sentences. It is classified into two sections:

1. **Sentence Tokenization and Word Tokenization:**
   Ex. Word Tokenization
   ```
   from nltk.tokenize import word_tokenize
   text = "Hello all, Welcome to Python Programming"
   tok_word = word_tokenize(text)
   print("Tokenized Words : \n",tok_word)
   ```

   Ex. Sentence Tokenization
   ```
   from nltk.tokenize import sent_tokenize
   text = "Hello all, Welcome to Python Programming.This is Data Analytics
           Programming.Sentence Tokenization and Word Tokenization."
   tok_sen = sent_tokenize(text)
   print("Tokenized Sentences : \n", tok_sen)
   ```
2. **Frequency Distribution:** The frequency distribution helps to understand how many words have occurred how many times in the given textual data.
   ```
   from nltk.probability import FreqDist
   from nltk.tokenize import word_tokenize
   text = "apple banana apple mango banana apple"
   ```

```
# Tokenize the text
words = word_tokenize(text)
# Create Frequency Distribution
fd = FreqDist(words)
print(fd)
#To find most common words using Frequency Distribution
print(fd.most_common())
print(fd.max())
```

Output:

```
<FreqDist with 3 samples and 6 outcomes>
[('apple', 3), ('banana', 2), ('mango', 1)]
apple
```

Meaning:

There are 3 unique items (distinct words) in data

Unique words = apple, banana, mango

There are **6 total words (including repetition)** in your data.

Count them = apple (3 times) ,banana (2 times) mango (1 time)=6

3. **Stop words** : Stop words are considered as Noise in textual data.
   For example, if text is containing words such as is, are, am, a, this, the, an etc. then they are treated as stop words.

```
from nltk.corpus import stopwords
stop_words_data=set(stopwords.words("english"))
print(stop_words_data)
```

4. **Removing Stop words:**

```
from nltk.tokenize import word_tokenize
from nltk.corpus import stopwords
text = "Hello all, Welcome to Python Programming.This is Data Analytics
Programming for removing stop word from Sentence"
tokenized_words=word_tokenize(text)
# It will find the stowords in English language.
```

```
stop_words_data=set(stopwords.words("english"))
# Create a stopwords list to filter it from original text
filtered_words_list=[]
for words in tokenized_words:
    if words not in stop_words_data:
        filtered_words_list.append(words)
print("Tokenized Words : \n",tokenized_words,"\n")
print("Filtered Words : \n",filtered_words_list,"\n")
```

5. **Stemming:** Stemming is a process of linguistics normalization to reduce words to their word root or divide the derivational affixes.
   For example: writing, wrote, written can stemmed or reduced as write.

```
import nltk
from nltk.stem import PorterStemmer
from nltk.tokenize import word_tokenize
# Create stemmer object
stemmer = PorterStemmer()
# Sample text
text = "running runs runner easily fairly cats studies studying"
# Tokenize text
words = word_tokenize(text)
# Apply stemming
stemmed_words = [stemmer.stem(word) for word in words]
print("Original Words:", words)
print("Stemmed Words:", stemmed_words)
```

6. **Lemmatization:** Lemmatization is a process of removing words to their base words which is linguistically correct lemmas. lemmatization needs **Part of Speech (POS)** for correct results.
   For example: "Runs" word will be lemmatized to "run".

   Example 1:

```
import nltk
from nltk.stem import WordNetLemmatizer
```

```
from nltk.tokenize import word_tokenize
# Create lemmatizer object
lemmatizer = WordNetLemmatizer()
text = "running runs better cats studies studying"
# Tokenize text
words = word_tokenize(text)
# Lemmatize words
lemmatized_words = [lemmatizer.lemmatize(word) for word in words]
print("Original Words:", words)
print("Lemmatized Words:", lemmatized_words)
```

Example 2:

```
# Lemmatization
from nltk.stem.wordnet import WordNetLemmatizer
lemmatizer=WordNetLemmatizer()
word_text="running"
print("Lemmatized Word : ",lemmatizer.lemmatize(word_text,"v"))
```

Set A

1. Consider any text paragraph. Preprocess the text to remove any special characters and digits. Generate the summary using extractive summarization process.

   In this program:

   1. Removes special characters and digits
   2. Tokenizes sentences
   3. Removes stopwords.
   4. Scores sentences based on word frequency
   5. Generates an extractive summary

```
import nltk
import re
import heapq
from nltk.corpus import stopwords
```

```
from nltk.tokenize import word_tokenize, sent_tokenize
text = " Text Summarization is an important application of Natural
Language Processing. It is used to reduce large amounts of text into
shorter versions while preserving the main meaning of the content.
There are two main types of summarizations: extractive and
abstractive summarization."
# Remove special characters and digits
clean_text = re.sub(r'[^a-zA-Z\s]', '', text)
print(clean_text)
#Tokenize into sentences
sentences = sent_tokenize(clean_text)
print(sentences)

# Word tokenization and stopword removal
stop_words = set(stopwords.words('english'))
word_frequencies = {}
for word in word_tokenize(clean_text.lower()):
    if word not in stop_words:
        if word not in word_frequencies:
            word_frequencies[word] = 1
        else:
            word_frequencies[word] += 1

# Normalize word frequencies
max_frequency = max(word_frequencies.values())
for word in word_frequencies:
    word_frequencies[word] /= max_frequency

#  Score sentences
sentence_scores = {}

for sentence in sentences:
    for word in word_tokenize(sentence.lower()):
        if word in word_frequencies:
            if sentence not in sentence_scores:
```

```
            sentence_scores[sentence] = word_frequencies[word]
        else:
            sentence_scores[sentence] += word_frequencies[word]

# Select top 2 sentences as summary
summary_sentences = heapq.nlargest(2, sentence_scores,
key=sentence_scores.get)
summary = ' '.join(summary_sentences)
print("Original Text:\n", text)
print("\nCleaned Text:\n", clean_text)
print("\nSummary:\n", summary)
```

2. Consider any text paragraph. Remove the stopwords. Tokenize the paragraph to extract words and sentences. Calculate the word frequency distribution and plot the frequencies.

```
import nltk
import string
import matplotlib.pyplot as plt
from nltk.corpus import stopwords
from nltk.tokenize import word_tokenize, sent_tokenize
from nltk.probability import FreqDist

# Sample paragraph
text = """
Natural Language Processing is a fascinating field of artificial
intelligence.
It helps computers understand, interpret, and generate human language.
NLP is widely used in chatbots, machine translation, sentiment analysis,
and many other applications.
"""

# Sentence Tokenization
```

```
sentences = sent_tokenize(text)
print("Sentences:")
print(sentences)

# Word Tokenization
words = word_tokenize(text)
print("\nWords:")
print(words)

# Remove Stopwords & Punctuation
stop_words = set(stopwords.words('english'))
filtered_words = [
    word.lower() for word in words
    if word.lower() not in stop_words and word not in string.punctuation
]

print("\nFiltered Words (No Stopwords):")
print(filtered_words)

#  Frequency Distribution
freq_dist = FreqDist(filtered_words)

print("\nWord Frequency Distribution:")
print(freq_dist)

#  Plot Frequency Distribution
plt.figure()
freq_dist.plot(10)
plt.title("Top 10 Word Frequencies")
plt.show()
```